

DEEP LEARNING, TOLD SIMPLY

Book 2 of 6

WRITTEN BY

Prof. Saurabh

One Tiny Brain Cell

How a brain cell works, how to copy it in maths, and the first time a machine learns by itself

Nova must make one small decision: wake up, or stay asleep? Asha looks inside a real brain cell, builds an artificial one, sets it by hand to be an AND gate and an OR gate, and then lets it learn a NOT gate on its own. Then she meets the one small puzzle a single cell can never solve. This book follows Module 2 of the Deep Neural Network course, slide by slide, in plain words.



What is inside

Book 2 covers Module 2: Artificial Neural Network and the Perceptron. Six short chapters. Read them in order; every chapter uses the one before it.

1	How a brain learns	3
	A brain is billions of slow, simple cells, all working at the same time, and it learns by linking things that happen together.	
2	One artificial cell	7
	Numbers come in, each one is multiplied by how much it matters, the results are added up, and a simple rule turns the total into one answer.	
3	Many cells: networks and connectionism	12
	Wire many simple neurons together in layers, and all the knowledge lives in the wires.	
4	The perceptron and logic gates	16
	A neuron with a hard yes/no threshold is called a perceptron, and with the right weights it can be an AND gate or an OR gate.	
5	How a perceptron learns	21
	Show it an example; if it is wrong, nudge the weights towards the right answer; repeat until nothing changes.	
6	The XOR wall	26
	One perceptron can only draw one straight line, and some very small problems cannot be cut by a straight line.	

How to read this book

Each chapter follows the same rhythm: a little story, one big idea, an everyday picture of it, the maths in small steps (first plain, then in matrix form), a worked example with real numbers, and a short check. The coloured boxes tell you what kind of thing you are reading:

-  **Story** — Asha and Nova, the thread through all six books
-  **Think of it like...** — a real-life picture of the idea
-  **Maths, step by step** — plain maths → matrix form
-  **On your slides** — the exact notation your deck uses
-  **Spark** — a deeper idea for the curious
-  **Big idea** — the one thing to remember
-  **Stop and think** — a question, then **Our answer**
-  **Worked example** — real numbers, one line per step
-  **Careful** — common mistakes
-  **Check yourself** — quick questions with answers

How a brain learns

A brain is billions of slow, simple cells, all working at the same time, and it learns by linking things that happen together.

On your slides: **Module 2, slides 5–8** · How do humans learn · Observation: the brain · Interconnected neurons · Biological neuron structure

STORY

Asha has 500 sound clips, each marked "Nova" or "not Nova". Now she needs something that can learn from them. "Where do I even start?" she asks. Ravi leans in the doorway. "Copy a brain. That is what everyone says." Asha laughs. "A whole brain has ten billion cells. I will start with one." She opens her biology notes to a drawing of a single brain cell. Nova's tiny screen shows a 0. It is waiting for a reason to show a 1.

BIG IDEA

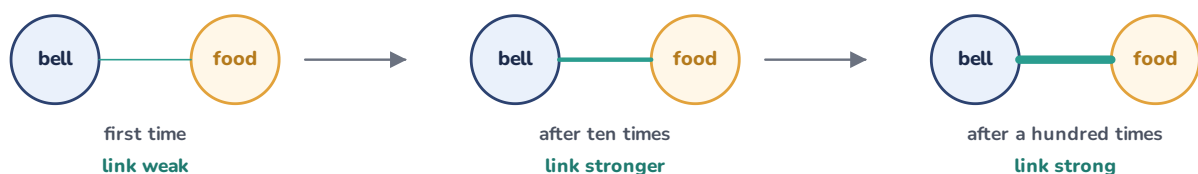
A brain learns by **association**: things that happen together get linked. It is built from about 10^{10} simple cells called **neurons**. Each cell is slow, but they all work at once. That is why the brain is fast. Every neuron has the same four parts, and Chapter 2 copies them in maths.

THINK OF IT LIKE...

The smell of onions frying makes you hungry before you see any food. Nobody taught you that rule. The smell and the meal simply came together, hundreds of times, and your brain linked them. Learning by association means: build a link between things that keep showing up together, and make the link stronger each time.

How do humans learn?

Your slide starts with a simple sentence: *it begins with the brain*. Humans learn, solve problems, recognise patterns, create, think deeply, meditate, and much more. All of it happens in the brain. And the slide gives one short answer to *how* we learn: **humans learn through association**. The old name for this idea is *associationism*.



Things that happen together get linked. The link is the memory.

Figure 1.1 Association. A bell rings and food arrives. The first time, the link between "bell" and "food" is weak. After many repeats it is strong. Now the bell alone makes you think of food. Notice that the *link* is what changed, not the bell or the food. Keep this picture: in every book that follows, learning means changing the strength of links.

Looking at the brain

Your slide lists a few facts about the brain. Read them slowly. They explain why neural networks are built the way they are.

Fact (from the slide)	In plain words
The brain is a mass of interconnected neurons	Lots of small cells, wired to each other.
Number of neurons $\approx 10^{10}$	Ten billion cells. A 1 followed by ten zeros.
Connections per neuron $\approx 10^4$ to 10^5	Each cell is linked to ten thousand to a hundred thousand other cells.
Neuron switching time ≈ 0.001 second	One cell takes about a millisecond to "fire". That is slow. A computer chip switches a billion times a second.
Scene recognition time ≈ 1 second	You look at a room and know what is in it in about one second.
100 inference steps doesn't seem like enough	See below. This is the surprising part.
Lot of parallel computation	So the brain must do very many things at the same time.

WORKED EXAMPLE: WHY THE BRAIN MUST WORK IN PARALLEL

Let us put two of the numbers together. One neuron takes about 1 millisecond. Recognising a scene takes about 1 second.

- 1 second = 1000 milliseconds.
change both times into the same unit
- So a chain of neurons, each waiting for the one before, can be at most about 1000 cells long.
 $1000 \text{ ms} \div 1 \text{ ms per step}$
- Real cells need several switchings to pass a useful signal on. Your slide rounds this down to **about 100 steps in a row**.
a fair, rough number
- A computer program that recognises a face needs millions of steps one after another. 100 is nowhere near enough.
compare with what a step-by-step program needs
- So the brain cannot be working step by step. It must do a huge number of small things **at the same time**. This is called **parallel computation**.
the only way the numbers fit

Bonus count: 10^{10} neurons $\times 10^4$ links each $\approx 10^{14}$ links. That is a hundred trillion links, and every one of them can be adjusted.

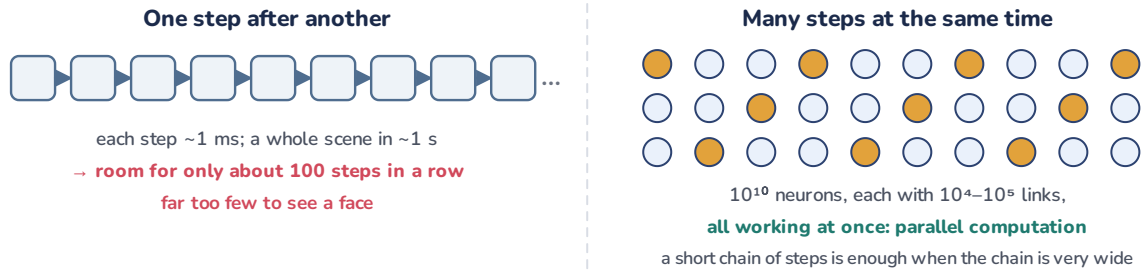


Figure 1.2 Left: steps done one after another. With 1 ms per step and 1 s in total, the chain can only be about 100 steps long. Right: ten billion slow cells working at once. A short chain is enough when the chain is very wide. Neural networks copy this: many simple units, all computing at the same time.

? STOP AND THINK

Ravi says: "One cook can make a thousand rotis. It just takes longer. Why does the brain need so many cells at once?"

✓ OUR ANSWER

Because the answer is needed in one second. One cook, at one roti a minute, cannot make a thousand rotis in one minute. A thousand cooks can. The brain is the thousand cooks. Each cell is slow, but there are ten billion of them, and they do not wait for each other.

Many in, many out

Your slide "Brain: Interconnected Neurons" shows a tangle of cells. Two sentences under it are the whole point:

- **Many neurons connect in to each neuron.** A cell listens to thousands of others.
- **Each neuron connects out to many neurons.** A cell's one answer is sent on to thousands of others.

So one cell is a small meeting point: many signals in, one signal out, copied to many places. Hold on to "many in, one out". The artificial neuron in Chapter 2 has exactly that shape.

The parts of one brain cell

A brain cell is called a **neuron**. Your slide names four parts.

Part	Job (from the slide)	In plain words
Dendrites	Receive signals from other neurons	The many ears of the cell.
Cell body (soma)	Processes incoming signals	Adds them up and decides whether the total is big enough to fire.
Axon	Transmits the output signal	One long wire that carries the cell's single answer away.
Synapses	Connection points between neurons	Where one cell's axon meets the next cell's dendrite. Each synapse has a strength, and the strength can grow or shrink.

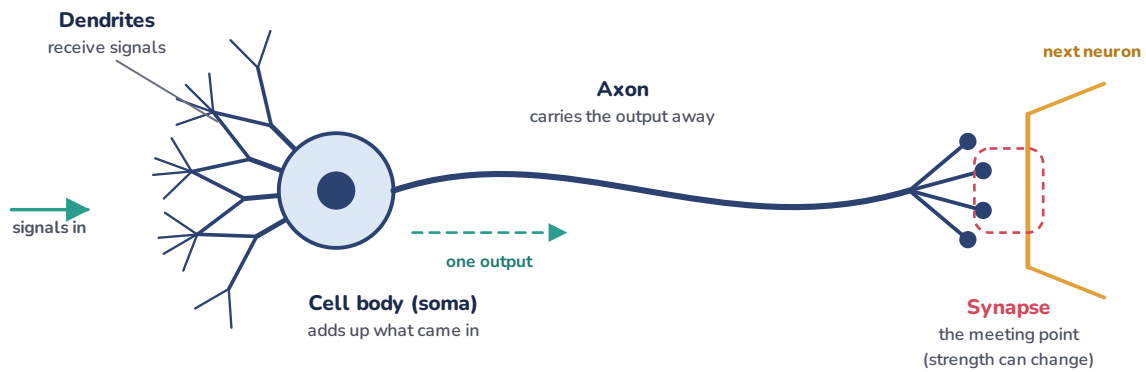


Figure 1.3 One biological neuron. Signals come in through the dendrites. The cell body adds them up and decides whether to fire. One signal then travels along the axon to the next cell. The meeting point is a synapse (dashed red box). Its strength can change, and that is where learning happens.

ON YOUR SLIDES

Slide 5 ends with "Humans learn through association [Refer to Associationism]". Slide 6, "Observation: The Brain", is the table above, word for word, with $10^{(4 \text{ to } 5)}$ written for the connections. Slide 7 is the tangle picture with the two "many in / many out" sentences. Slide 8, "Biological Neuron Structure", names dendrites, cell body (soma), axon and synapses with the same one-line jobs.

CAREFUL

All the brain numbers are rough. "About 10^{10} " means "somewhere around ten billion". What matters is the size of the numbers, not the digits. Also: artificial neural networks are *inspired* by the brain, not copies of it. Chapter 2 keeps only the simplest parts.

CHECK YOURSELF

1. In one word, how do humans learn, according to your slide? **Association: things that happen together get linked.**
2. About how many neurons are in a brain, and how many connections does each have? **About 10^{10} neurons; about 10^4 to 10^5 connections each.**
3. One cell takes 1 ms and a scene takes 1 s. What does this tell us? **Only about 100 steps can happen one after another, so the brain must work in parallel.**
4. Name the four parts of a neuron and their jobs. Where does learning happen? **Dendrites receive; cell body (soma) processes; axon transmits the output; synapses connect neurons. Learning changes the synapse strengths.**

IN ONE BREATH

- Humans learn by association: link things that happen together, and strengthen the link each time.
- 10^{10} slow neurons, 10^4 – 10^5 links each, 1 ms per switch, 1 s per scene $\Rightarrow$ only ~ 100 serial steps $\Rightarrow$ massive parallel computation.
- Many in, one out. Dendrites receive, the soma adds up and decides, the axon sends, the synapse connects and its strength can change.

One artificial cell

Numbers come in, each one is multiplied by how much it matters, the results are added up, and a simple rule turns the total into one answer.

On your slides: **Module 2, slides 10–13** · What is an artificial neuron · Artificial neuron N · Mathematical computation · Biological vs artificial neurons

STORY

Asha gives Nova three tiny sensors: is the room dark? Is someone near? Is there a loud noise? Each sensor answers 1 for yes and 0 for no. She wants one decision from the three answers: wake up, or not. "Not every sensor matters the same," she tells Ravi. "Someone near matters most. A loud noise should actually keep Nova asleep. It's probably a door slamming." She writes a number next to each sensor. Nova's screen shows the total, and then a single 1.

BIG IDEA

An **artificial neuron** is a tiny calculator. It takes in numbers $x_1, \dots, x_n$. It multiplies each by a **weight** w_i that says how much that input matters. It adds the results, plus one extra number called the **bias** b . Then an **activation function** f turns the total into the output $\hat{y}$. The weights and bias are the cell's knowledge.

THINK OF IT LIKE...

A final mark in a course. Your quiz, assignment and exam marks are the *inputs*. Each has a *weightage*: the exam counts for more than a quiz. Multiply each mark by its weightage and add. Grace marks added to everyone are the *bias*. Then a rule decides: 40 or above is a pass. That rule is the *activation function*. Marks in, weightages, add, rule, one answer out. That is the whole neuron.

What is an artificial neuron?

Your slide's definition: an artificial neuron is a **processing element inspired by how the brain works**. Three facts sit under it, each copied from Chapter 1.

- Like a brain cell, each artificial neuron **does some computation**. Many in, one out.
- Each neuron is **interconnected** to other neurons, as in the brain's tangle.
- The interconnections **store the knowledge** it learns. The knowledge is stored as **parameters**, that is, as numbers. In the brain the synapse strength changes; here a number changes.

The neuron N, part by part

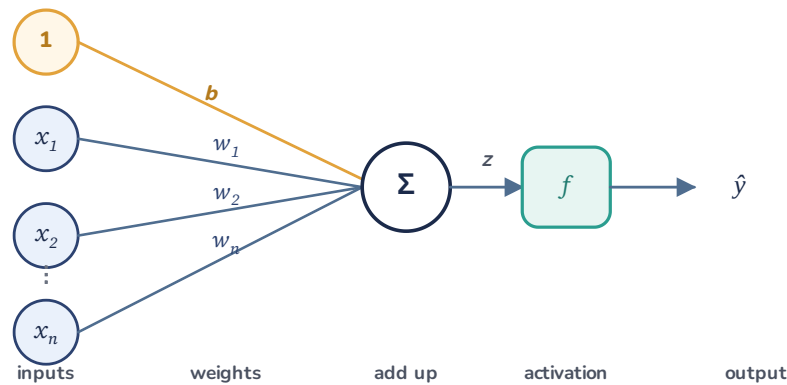


Figure 2.1 The artificial neuron from your slide. Each input x_i travels along a line that carries a weight w_i . The circle Σ adds everything up. The extra input at the top is always 1; its weight is the bias b . The box f is the activation function. Notice the four parts of Figure 1.3 again: inputs (dendrites), weights (synapse strengths), adding up (cell body), one output (axon).

Your slide says three things about the neuron N . Read each one with the picture.

- The processing neuron N performs a **weighted sum**. "Weighted" means each input is multiplied by its own number before adding.
- For each feature x_i , the weight w_i shows the **importance** of that feature. A big weight means "this input matters a lot". A weight near zero means "ignore this input". A negative weight means "this input pushes the answer *down*".
- N generates **one numerical output** $\hat{y}$ (say "y hat"). It means "the cell's guess for y ".

Σ MATHS, STEP BY STEP

PLAIN MATHS

Take three inputs. Multiply each by its weight, add them, add the bias. Call the total z :

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

For n inputs, the same thing with a sum sign (Σ means "add up all of these"):

$$z = \sum_{i=1}^n w_i x_i + b$$

Then pass z through the activation function f :

$$\hat{y} = f(z) = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

In this book f is a simple rule: output 1 if $z \geq 0$, else 0. Later books use other rules.

MATRIX FORM

Put the inputs in one column $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top$ and the weights in one column $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top$. Both are $n \times 1$. The little $^\top$ (transpose) turns a row into a column, or a column into a row.

The weighted sum is then one row times one column:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad \hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$$

$\mathbf{w}^\top$ is $1 \times n$, $\mathbf{x}$ is $n \times 1$, so $\mathbf{w}^\top \mathbf{x}$ is 1×1 : one number, the same z . Computers run this form: one multiplication of a row by a column, instead of a loop over i .

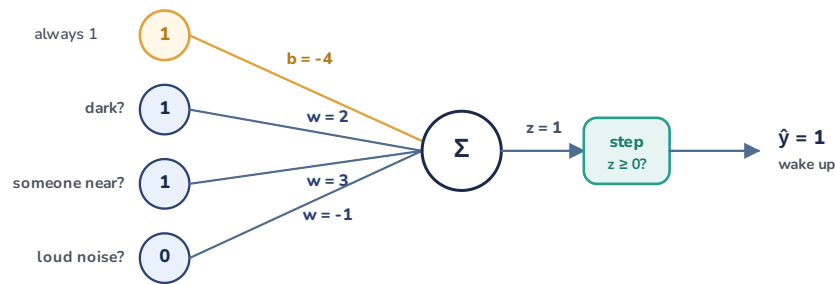


Figure 2.2 Case 1 of the worked example below, drawn on the neuron. Follow the numbers left to right: inputs 1, 1, 0 and the always-on 1; weights 2, 3, -1 and bias -4; total $z = 1$; the step rule gives $\hat{y} = 1$.

WORKED EXAMPLE: NOVA'S WAKE-UP CELL

Inputs: $x_1 = \text{dark?}$, $x_2 = \text{someone near?}$, $x_3 = \text{loud noise?}$. Each is 1 for yes, 0 for no. Asha sets the weights by hand: $w_1 = 2$, $w_2 = 3$, $w_3 = -1$, and the bias $b = -4$. The rule is: $\hat{y} = 1$ (wake up) if $z \geq 0$, else $\hat{y} = 0$ (stay asleep).

Case 1: it is dark, someone is near, no noise: $x = (1, 1, 0)$.

- 1 $z = 2 \times 1 + 3 \times 1 + (-1) \times 0 + (-4)$
each input times its weight, plus the bias
- 2 $z = 2 + 3 + 0 - 4 = 1$
do the multiplications, then add
- 3 $1 \geq 0$, so $\hat{y} = 1$. Nova wakes up.
apply the activation rule

Case 2: it is dark, nobody is near, no noise: $x = (1, 0, 0)$.

- 1 $z = 2 \times 1 + 3 \times 0 + (-1) \times 0 + (-4) = 2 + 0 + 0 - 4 = -2$
- 2 $-2 < 0$, so $\hat{y} = 0$. Nova stays asleep.

Look at what the numbers say. Darkness alone (2) cannot beat the bias (-4). Darkness plus a person ($2 + 3 = 5$) can. And the noise weight is negative, so a loud noise makes waking up *harder*: with $x = (1, 1, 1)$, $z = 2 + 3 - 1 - 4 = 0$, just enough to wake. The bias -4 sets how hard it is to wake Nova at all.

? STOP AND THINK

Ravi asks: "Why do we need the bias? The weights already say what matters."

✓ OUR ANSWER

The bias sets the cell's starting mood. When every input is 0, the sum of $w_i x_i$ is 0, so $z = b$. With $b = -4$, Nova sleeps when nothing is happening. With $b = +1$, Nova would wake up for no reason. Weights say how much each input pushes; the bias says where the pushing starts. In the marks analogy, it is the grace marks: it moves everyone up or down before the pass rule is applied.

Biological versus artificial

Your slide puts the two side by side. The left column is Chapter 1. The right column is this chapter.

Biological neuron	Artificial neuron	In plain words
Complex biochemical processes	Simple mathematical model	A cell is chemistry; ours is a multiply-and-add.
Analog signal processing	Digital computation	A cell's signals vary smoothly; ours are numbers in a computer.
Adaptive synaptic strengths	Adjustable weights	The synapse strength becomes the weight w_i . Both can change.
Parallel processing	Parallel computation possible	Many cells at once; many neurons at once (on a GPU).
Learning through experience	Learning through algorithms	A brain learns by living; ours learns by a fixed procedure (Chapter 5).
Fault tolerant	Deterministic behavior	A brain keeps working when cells die; ours always gives the same output for the same input.

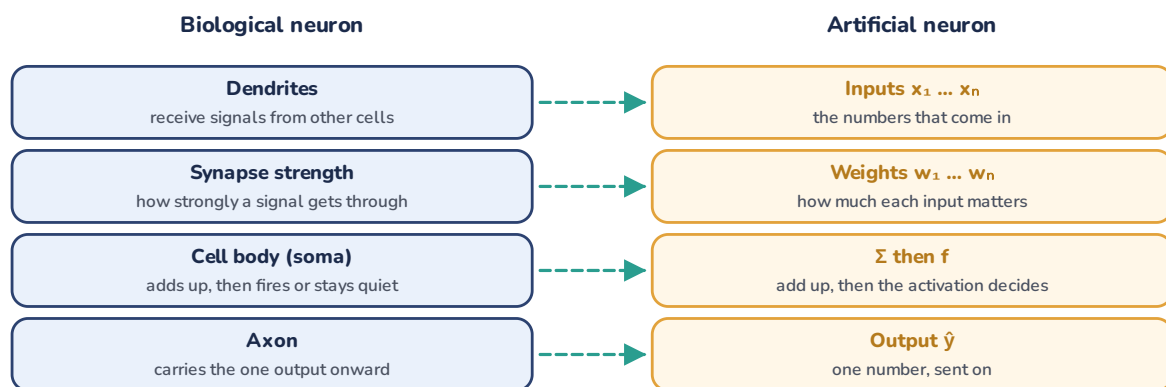


Figure 2.3 The part-by-part mapping. Dendrites become the inputs, synapse strengths become the weights, the cell body becomes "add up, then decide", and the axon becomes the single output. Notice that the only things that change during learning, on both sides, are the middle row: the strengths, or the weights.

Your slide ends with a **key insight**: artificial neurons are simplified mathematical abstractions that capture the essential computational properties of biological neurons. In plain words: we keep the shape (many in, weighted, added, one out) and throw away the chemistry.

ON YOUR SLIDES

Slide 10 is the definition and the picture of $x_1, \dots, x_n$ going into N and $\hat{y}$ coming out. Slide 11 adds the weights and the bias input "1" with weight b . Slide 12 gives equations (1) $z = \sum_{i=1}^n w_i x_i + b$ and (2) $\hat{y} = f(\sum_{i=1}^n w_i x_i + b)$, with a Σ circle and an $f(\cdot)$ box. Slide 13 is the comparison table and the key insight.

CAREFUL

Three different kinds of number appear in one neuron. The inputs x_i are *given* by the data. The weights w_i and bias b are *learned*: they are the parameters. The activation f is *chosen* by us and does not change during learning. Mixing these up is the most common mistake in the whole course.

🔧 TRY IT YOURSELF

Keep Asha's weights $w = (2, 3, -1)$ and $b = -4$. Someone is near, it is not dark, and there is a loud noise: $x = (0, 1, 1)$. Compute z and $\hat{y}$. ($z = 0 + 3 - 1 - 4 = -2$, so $\hat{y} = 0$: Nova stays asleep.)

✅ CHECK YOURSELF

1. What does the weight w_i mean? **The importance of input x_i : how much it pushes the total.**
2. Write the neuron's computation in two lines. $z = \sum_i w_i x_i + b$; $\hat{y} = f(z)$.
3. Where is the neuron's knowledge stored? **In the parameters: the weights and the bias.**
4. What is z when all inputs are zero? $z = b$, **the bias.**
5. In matrix form, what are the shapes in $\mathbf{w}^\top \mathbf{x}$? **$\mathbf{w}^\top$ is $1 \times n$, $\mathbf{x}$ is $n \times 1$, the result is one number.**

🧠 IN ONE BREATH

- Artificial neuron = weighted sum + bias, then an activation: $z = \sum_i w_i x_i + b$, $\hat{y} = f(z)$. In vector form $z = \mathbf{w}^\top \mathbf{x} + b$.
- Weight = importance of an input. Bias = where the pushing starts. Both are learned; the inputs are given; f is chosen.
- Dendrites → inputs, synapse strengths → weights, soma → add and decide, axon → one output. Keep the shape, drop the chemistry.

Many cells: networks and connectionism

Wire many simple neurons together in layers, and all the knowledge lives in the wires.

On your slides: **Module 2, slides 15–17, 19–20** · What are artificial neural networks · Properties · When to consider ANNs · Connectionist machines · Connectionism principles

📖 STORY

One cell gave Nova one yes/no from three sensors. But a sound clip is thousands of numbers, and "Nova" is made of many small patterns: a soft "No", a rising "va". "One cell cannot hold all that," Asha says. Ravi shrugs. "Then use two." Asha draws circles in columns and joins every circle to every circle in the next column. "Use dozens. In layers." She counts the lines. Nova's screen shows 36: the number of weights this little network would have to learn.

! BIG IDEA

An **artificial neural network (ANN)** is many artificial neurons wired together, usually in layers: an input layer, some hidden layers, and an output layer. Every wire carries a weight. The network's knowledge is stored in those weights, and learning means changing them. This view, that intelligence comes from simple units and their connections, is called **connectionism**.

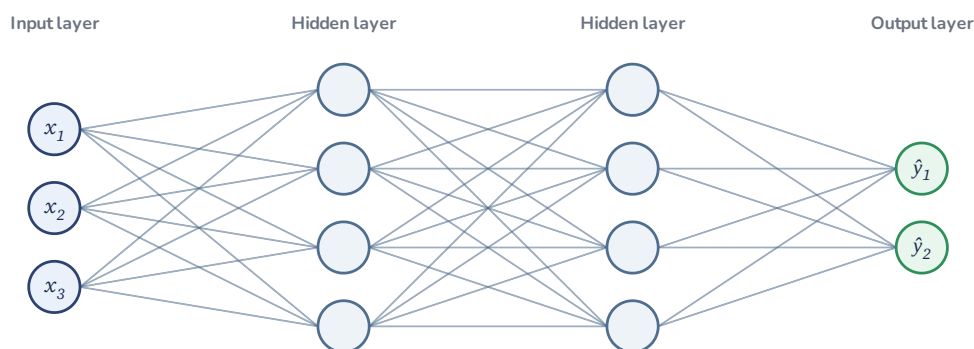
≈ THINK OF IT LIKE...

A big restaurant kitchen. Nobody there cooks a whole biryani alone. One person washes rice, one fries onions, one layers the pot, one checks the salt. Each job is simple. The dish comes from *how the jobs are connected*: who passes what to whom, and how much. Move the connections around and the same people make a different dish. In a network the neurons are the cooks, and the weights are the passing-along. The recipe is in the connections.

What is an artificial neural network?

Your slide says five things. Here is each one in plain words.

- A neural network is a **computational model inspired by the human brain**. "Computational model" just means a way of computing an answer from an input.
- **Interconnected neurons process information**. Each neuron does the small multiply-add-decide job from Chapter 2. Together they do something big.
- The interconnection can be **forward, recursive or a self loop** (see Figure 3.2). This course mostly uses forward.
- It **learns patterns from data through training**. Nobody types the weights in. The data sets them (Chapter 5 shows the first way to do this).
- It is **capable of approximating complex functions**. A function is any rule from input to output. "Sound clip → did someone say Nova?" is a very complex rule. A big enough network can come as close to it as we like.



Every line is one weight. The knowledge lives in the lines, not in the circles.

Figure 3.1 A small network, drawn like your slide: an input layer, two hidden layers, an output layer. Each circle is one Chapter 2 neuron. Each line is one weight. The middle layers are **hidden** because you never see their outputs; only the last layer's answer comes out. Count the lines: $3 \times 4 + 4 \times 4 + 4 \times 2 = 36$ weights.

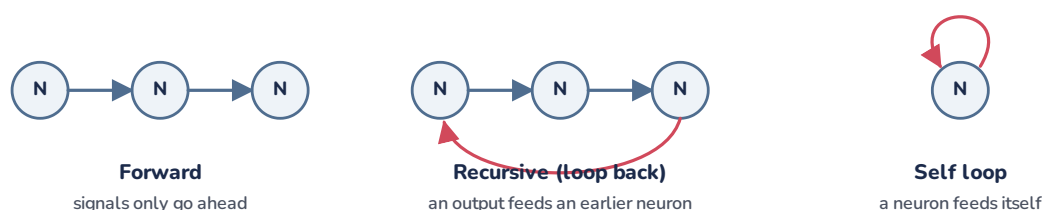


Figure 3.2 Three ways to connect neurons. **Forward**: signals only move on to the next layer. **Recursive**: an output is fed back to an earlier neuron, which gives a kind of memory (used for sequences later in the course). **Self loop**: a neuron feeds itself. Books 2–6 use forward networks only.

Properties of an artificial neural network

Your slide lists four. They are the brain facts of Chapter 1, copied.

Many neurons

Dozens to billions of simple units.

Many weighted interconnections

Every line has a weight; the lines carry the knowledge.

Highly parallel, distributed process

All neurons in a layer compute at once; no single neuron holds the answer.

Tuning weights automatically

Training sets the weights. Nobody types them in.

WORKED EXAMPLE: HOW MANY NUMBERS MUST THIS NETWORK LEARN?

Take the network in Figure 3.1: layers of 3, 4, 4 and 2 neurons, every neuron joined to every neuron in the next layer.

- 1 Input to first hidden layer: $3 \times 4 = 12$ weights.
3 inputs, each joined to 4 neurons
- 2 First hidden to second hidden: $4 \times 4 = 16$. Second hidden to output: $4 \times 2 = 8$.
- 3 Total weights: $12 + 16 + 8 = 36$. That is the number Nova showed.
- 4 Each of the $4 + 4 + 2 = 10$ neurons also has a bias, so $36 + 10 = 46$ parameters in all.
inputs are not neurons, so they have no bias

Real networks have millions of such numbers. Every one is set by training, not by hand.

When should you use a neural network?

Your slide gives a checklist. If most of these are true for your problem, a neural network is a good choice.

From the slide	In plain words	Nova's wake word?
Input is high-dimensional, discrete or real-valued (e.g. raw sensor input)	Each example is a long list of numbers, often straight from a sensor.	Yes: thousands of microphone readings per clip.
Possibly noisy data; data has lots of errors	The examples are messy. Some labels may even be wrong.	Yes: fans, traffic, coughs.
Output is discrete, real-valued, or a vector of values	The answer can be a category, a number, or a list of numbers.	Yes: one category, yes/no.
Form of target function is unknown	You have no formula for the rule you want. You only have examples.	Yes: there is no formula for "Nova".
Human readability (explainability) of the result is unimportant	You do not need to read off <i>why</i> the network answered as it did.	Yes: Asha only needs it to work.

The slide's examples are **speech recognition**, **image classification** and **financial prediction**. Check them against the table: long noisy inputs, no known formula, and nobody needs a written explanation for every answer.

Connectionist machines

Your slide gives the old name for this whole way of thinking. A **connectionist machine** is:

- a **network of processing elements**, called artificial neurons;
- the neurons are **interconnected** to form a network;
- the interconnection can be **forward, recursive or self loop**;
- **all world knowledge is stored in the connections** between these elements.

And the punch line: **neural networks are connectionist machines**. Look again at Figure 3.1. The circles are all the same. Only the lines differ. Everything the network knows, it knows through its lines.

Connectionism principles

Your slide boils the idea down to four core ideas.

Intelligence emerges from simple neurons

No single unit is clever. Many together are.

Knowledge is stored in connection weights

Not in the neurons. In the numbers on the lines.

Learning modifies connection strengths

To learn is to change weights.

Parallel distributed processing

Many units at once; knowledge spread across all of them.

? STOP AND THINK

Ravi says: "Each neuron only multiplies and adds. So where exactly is the cleverness?"

✓ OUR ANSWER

Nowhere in particular, and that is the point. It is in the pattern of weights across the whole network. Copy the trained weights into a fresh network, and the fresh one hears "Nova" too. Change the weights, and the same neurons hear something else. The cleverness is in the connections, not in the units. That is connectionism in one sentence.

📄 ON YOUR SLIDES

Slide 15 is the five-bullet definition next to the input / hidden layers / output layer picture. Slide 16 lists the four properties. Slide 17 is the "When to consider ANNs?" checklist with three examples. Slide 19, "Connectionist Machines", has the four bullets and "Neural networks are Connectionist machines". Slide 20 lists the four core ideas.

! CAREFUL

"Explainability is unimportant" is a condition for *choosing* a network, not a promise. Be ready for a model whose reasons are hard to read off. If a doctor or a judge must explain every decision, a network may be the wrong tool.

✓ CHECK YOURSELF

1. Name the three kinds of layer in an ANN. **Input layer, hidden layers, output layer.**
2. Name the three kinds of interconnection. **Forward, recursive, self loop.**
3. Where does a connectionist machine store its knowledge? **In the connections (the weights).**
4. Give two conditions from the slide that suggest using a neural network. **Any two of: high-dimensional input, noisy data, unknown target function, explainability unimportant.**
5. How many weights join a layer of 4 neurons to a layer of 2? $4 \times 2 = 8$.

🗨 IN ONE BREATH

- ANN = neurons in layers (input, hidden, output) joined by weighted lines; learns from data; can approximate very complex rules.
- Use one when inputs are long and noisy, the output is a category or number, no formula is known, and you do not need a written explanation.
- Connectionism: simple units, knowledge in the weights, learning changes the weights, everything runs in parallel.

The perceptron and logic gates

A neuron with a hard yes/no threshold is called a perceptron, and with the right weights it can be an AND gate or an OR gate.

On your slides: **Module 2, slides 22–31** · The perceptron · Representing logic gates · AND gate and its perceptron · OR gate and its perceptron

STORY

Asha keeps only two sensors: is the room dark (x_1)? Is someone near (x_2)? She wants a strict rule: wake up only if it is dark *and* someone is near. "That is an AND gate," says Ravi, who has done digital electronics. "You can buy one for two rupees." Asha shakes her head. "I don't want to buy it. I want one cell to *be* it." She writes three numbers on the whiteboard: $w_0 = -1$, $w_1 = 0.75$, $w_2 = 0.75$. She tries all four cases. Nova's screen shows 0, 0, 0, 1.

BIG IDEA

A **perceptron** is the simplest artificial neuron: a weighted sum plus a bias, then a hard **threshold**: output 1 if the sum is at least 0, else output 0 (or -1 in the classical version). Choosing the weights draws a straight line through the input space. The perceptron says 1 on one side of the line and 0 on the other. For AND and OR, such a line exists, so one perceptron is enough.

THINK OF IT LIKE...

A rope laid straight across a playground. Everyone on one side is "in", everyone on the other side is "out". The rope does not bend. Choosing the perceptron's weights is choosing where to lay the rope and which way it faces. The AND gate needs a rope that leaves only the corner (1,1) on the "in" side. The OR gate needs a rope that leaves only the corner (0,0) on the "out" side. Both are easy with one straight rope.

The perceptron

Your slide says: **perceptron = simplest form of artificial neuron**. It was introduced by **Rosenblatt in 1958**, the first date on the timeline in Book 1. It is exactly the neuron of Chapter 2 with one particular activation function: a threshold.

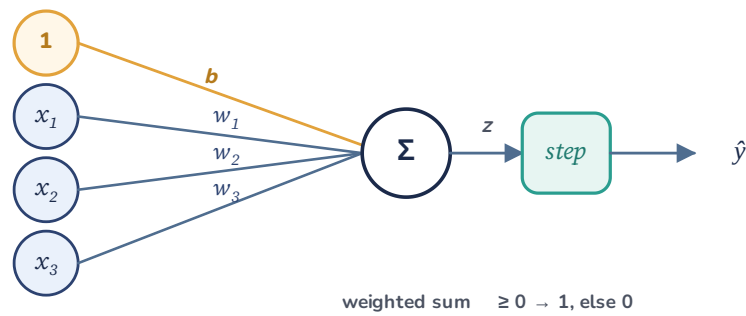


Figure 4.1 The perceptron, drawn as on your slide: inputs x_1, x_2, x_3 , weights w_1, w_2, w_3 , the bias b on an always-on input of 1, a weighted sum, then a step. Notice that only the last box is new: instead of a general f , the perceptron uses "1 if the sum is at least 0".

The slide writes the output rule like this:

$$\hat{y} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i + b \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Read it as: add up the weighted inputs and the bias. If the total is zero or positive, say 1. Otherwise say -1. The total itself is often called h on later slides. This rule is also written $\hat{y} = \text{sign}(h)$.

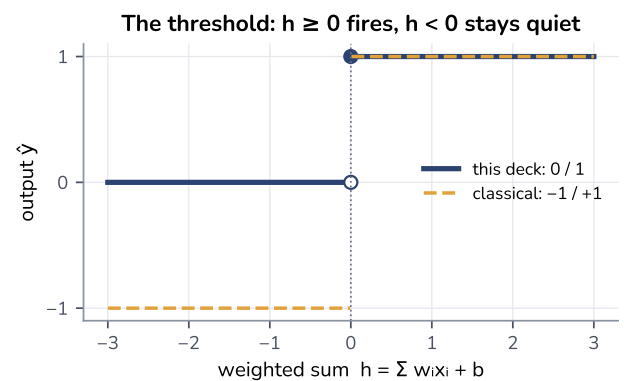


Figure 4.2 The threshold as a picture. Left of zero the output is low, right of zero it is high, and exactly at zero it is high. Solid: the 0/1 version used in this deck's examples. Dashed: the classical -1/+1 version.

PLUS ONE OR MINUS ONE? ZERO OR ONE?

Your slide 25 stops to make a small but important point. The classical perceptron (Rosenblatt's original) uses $\{-1, +1\}$ as outputs. Modern implementations and logic gates often use $\{0, 1\}$. **Both are valid.** The choice affects the bias and threshold calculation. The update rule in Chapter 5 multiplies by $(t - \hat{y})$. That difference is 1 in one system and 2 in the other. The slide's note says clearly: *the numerical examples that follow use binary values $\{0, 1\}$* . So from here on, low output means 0.

The same slide also states the goal of the next few pages: **any linear decision can be represented by a Boolean equation.** A "linear decision" is one straight rope. A "Boolean equation" is a rule built from AND, OR and NOT. The quest is to find out how to represent such a rule with a perceptron. So we test each logic gate one by one.

The AND gate

An AND gate answers 1 only when *both* inputs are 1. Your slide gives its table and asks: how do we represent the AND gate with a perceptron, and what are its parameters? With two inputs there are three parameters: w_0 (the bias, written as a weight on a fixed input $x_0 = 1$), w_1 and w_2 . The slide calls the inputs A and B and the output AB.

A	B	AB (AND)
0	0	0
0	1	0
1	0	0
1	1	1

The picture on the right shows the answer we are about to find. The four rows of the table are four points, and one straight line separates the single 1 from the three 0s.

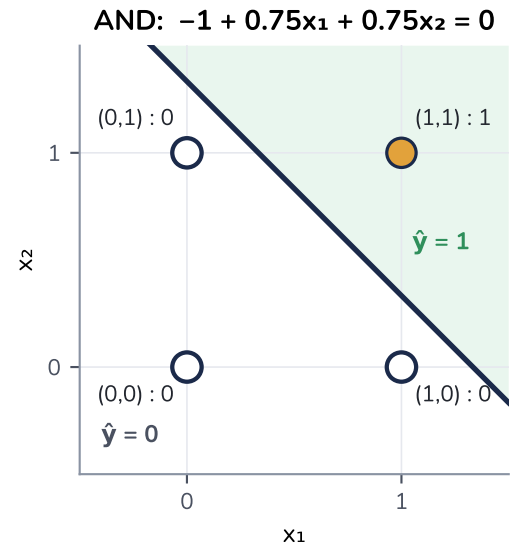


Figure 4.3 The AND perceptron as a picture, like your slide 28. Filled orange = target 1, hollow = target 0. The dark line is $-1 + 0.75x_1 + 0.75x_2 = 0$, the slide's "linear separation". The green side is where the perceptron says 1. Only (1,1) is on that side.

Σ MATHS, STEP BY STEP

PLAIN MATHS

The slide writes the perceptron equation with the bias as w_0 times a fixed input $x_0 = 1$:

$$h = w_0x_0 + w_1x_1 + w_2x_2 = w_0 + w_1x_1 + w_2x_2$$

Then $\hat{y} = 1$ if $h \geq 0$, else $\hat{y} = 0$. For the AND table, each of the four rows becomes one condition on h :

x_1	x_2	t	needs
0	0	0	$w_0 + 0 + 0 < 0$
0	1	0	$w_0 + 0 + w_2 < 0$
1	0	0	$w_0 + w_1 + 0 < 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

Any w_0, w_1, w_2 that satisfy all four lines is a solution. The slide's choice: $w_0 = -1, w_1 = 0.75, w_2 = 0.75$.

MATRIX FORM

Put the fixed 1 in front of the inputs: $\tilde{\mathbf{x}} = [1, x_1, x_2]^T$, and the weights in $\mathbf{w} = [w_0, w_1, w_2]^T$. Then

$$h = \mathbf{w}^T \tilde{\mathbf{x}}, \quad \hat{y} = \begin{cases} 1 & h \geq 0 \\ 0 & h < 0 \end{cases}$$

The boundary between "1" and "0" is where $h = 0$:

$$w_0 + w_1x_1 + w_2x_2 = 0$$

This is the equation of a straight line in the (x_1, x_2) plane. Solve for x_2 :

$$x_2 = -\frac{w_0 + w_1x_1}{w_2}$$

For the AND solution: $x_2 = \frac{1 - 0.75x_1}{0.75} = 1.333 - x_1$. It cuts each axis at 1.333, just past the corner (1,1). Points above the line give $\hat{y} = 1$.

WORKED EXAMPLE: CHECKING THE AND SOLUTION

Take $w_0 = -1, w_1 = 0.75, w_2 = 0.75$ and compute $h = w_0 + w_1x_1 + w_2x_2$ for every row.

- 1 (0,0): $h = -1 + 0.75 \times 0 + 0.75 \times 0 = -1$. $-1 < 0$, so $\hat{y} = 0$. Target 0. ✓
- 2 (0,1): $h = -1 + 0 + 0.75 = -0.25$. $\hat{y} = 0$. Target 0. ✓
- 3 (1,0): $h = -1 + 0.75 + 0 = -0.25$. $\hat{y} = 0$. Target 0. ✓
- 4 (1,1): $h = -1 + 0.75 + 0.75 = 0.5$. $0.5 \geq 0$, so $\hat{y} = 1$. Target 1. ✓

All four rows match. One perceptron is an AND gate. Notice *why* it works: one input alone gives 0.75, which cannot beat the bias of -1 . Two inputs together give 1.5, which can.

The OR gate

An OR gate answers 1 when *at least one* input is 1. Only (0, 0) gives 0. Your slide calls the output A+B and asks the same question: what are w_0, w_1, w_2 ? The four rows now give these conditions:

x_1	x_2	t	needs
0	0	0	$w_0 + 0 + 0 < 0$
0	1	1	$w_0 + 0 + w_2 \geq 0$
1	0	1	$w_0 + w_1 + 0 \geq 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

One solution is $w_0 = -1, w_1 = 2, w_2 = 2$.

Compare with AND. The bias is the same, -1 . The input weights are bigger, 2 instead of 0.75. Now a *single* input (2) already beats the bias, so one yes is enough. That is exactly what OR means. The line $-1 + 2x_1 + 2x_2 = 0$ cuts each axis at 0.5, between (0,0) and the other three corners.

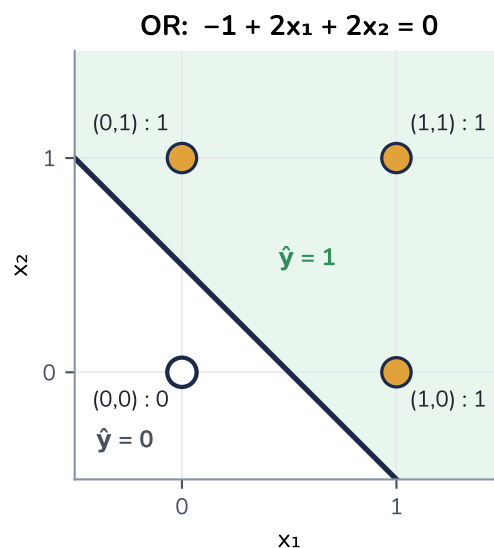


Figure 4.4 The OR perceptron, like your slide 31. Three points are on the green side; only (0,0) is left out. Notice that the line has moved towards the origin.

🔍 TRY IT YOURSELF

Check the OR solution the way we checked AND: compute $h = -1 + 2x_1 + 2x_2$ for the four rows and apply the threshold. ($h = -1, 1, 1, 3$, so $\hat{y} = 0, 1, 1, 1$. All four match.)

? STOP AND THINK

Ravi asks: "Is $w_0 = -1, w_1 = w_2 = 0.75$ the *only* AND perceptron?"

✓ OUR ANSWER

No. The slide says "*one* solution is". Try $w_0 = -1.5, w_1 = w_2 = 1$: the four totals are $-1.5, -0.5, -0.5, 0.5$, which also gives 0, 0, 0, 1. Any rope that leaves only the corner (1,1) on the "in" side works, and there are infinitely many such ropes. Chapter 6 comes back to this.

📄 ON YOUR SLIDES

Slides 22–23 draw the perceptron with x_1, x_2, x_3 and give $\hat{y} = 1$ if $\sum_{i=1}^n w_i x_i + b \geq 0$, else -1 . Slide 25 is the $\{-1, +1\}$ versus $\{0, 1\}$ note. Slides 26–28 are the AND table (columns A, B, AB), the "Goal: find parameters w_0, w_1, w_2 " table with the four conditions on h , the line "One solution is $w_0 = -1, w_1 = 0.75, w_2 = 0.75$ ", and the picture with the "linear separation" line. Slides 29–31 do the same for OR (columns A, B, A+B) with $w_0 = -1, w_1 = 2, w_2 = 2$. On those slides the perceptron equation is written $\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$; there, $\hat{y}$ means the total *before* the threshold, which the table calls h .

! CAREFUL

The threshold is "greater than *or equal to* zero". A total of exactly 0 gives output 1, not 0. This matters in Chapter 5, where the first total is exactly 0 and the perceptron says 1. Also keep the sign system straight: this deck's examples use 0/1 outputs, so "wrong by one" is always $t - \hat{y} = \pm 1$.

✓ CHECK YOURSELF

1. Who introduced the perceptron, and when? **Rosenblatt, 1958.**
2. Write the perceptron's output rule with 0/1 outputs. **$\hat{y} = 1$ if $w_0 + \sum_i w_i x_i \geq 0$, else 0.**
3. For AND with $w = (-1, 0.75, 0.75)$, what is h at (1, 0)? **-0.25 , so $\hat{y} = 0$.**
4. Why do the OR weights need to be bigger than the AND weights? **One input alone must beat the bias for OR; for AND it must not.**
5. What does the equation $w_0 + w_1 x_1 + w_2 x_2 = 0$ describe? **The straight boundary line between the 1 side and the 0 side.**

🗨 IN ONE BREATH

- Perceptron (Rosenblatt, 1958) = weighted sum + bias, then a threshold at 0. Classical outputs ± 1 ; this deck's examples use 0/1.
- Each truth-table row gives one condition on $h = w_0 + w_1 x_1 + w_2 x_2$. AND: $w = (-1, 0.75, 0.75)$. OR: $w = (-1, 2, 2)$. Many other solutions exist.
- $h = 0$ is a straight line; the perceptron says 1 on one side and 0 on the other.

How a perceptron learns

Show it an example; if it is wrong, nudge the weights towards the right answer; repeat until nothing changes.

On your slides: **Module 2, slides 33–39** · Perceptron learning algorithm · Convergence theorem · Perceptron learning for the NOT gate, epochs 1–3

STORY

"You set the AND weights yourself," Ravi says. "That is not learning. That is typing." Asha agrees. She picks the smallest gate of all, NOT: one input, one output, the opposite of the input. Nova's light sensor gives $x_1 = 1$ when the room is bright. Nova should be awake, $t = 1$, exactly when it is *not* bright. Asha sets both weights to 0 and refuses to touch them again. She shows Nova the two examples, again and again. Nova's screen shows the weights: 0, 0. Then -1, -1. Then 0, -1. Then it stops changing.

BIG IDEA

The **perceptron learning algorithm**: go through the examples one at a time. Compute the guess $\hat{y}$. If it equals the target t , change nothing. If not, move every weight a small step in the direction of its input: $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$, and the bias $b \leftarrow b + \eta(t - \hat{y})$. Repeat over all examples until a whole pass makes no change. If a separating line exists, this procedure is guaranteed to find one.

THINK OF IT LIKE...

Setting the volume on a radio by ear. Too quiet? Turn the knob up one notch. Too loud? Down one notch. Just right? Leave it. In the learning rule, $(t - \hat{y})$ is the ear. It is +1 when the perceptron said 0 but should have said 1 (too quiet). It is -1 when it said 1 but should have said 0 (too loud). It is 0 when it was right. η is the size of one notch. And x_i says *which* knobs to turn: only the weights of inputs that were switched on. An input that was 0 played no part, so its knob is left alone.

The learning algorithm

Your slide states the goal first: **find weights that correctly classify the training data**, given as (input, output) pairs (x, t) . Then the algorithm, in four steps.

1. **Initialise the learning rate** $\eta = 0.1$. η (say "eta") is the size of each correction. The slide's example later uses $\eta = 1$.
2. **Initialise the weights randomly**: $w_i = \text{random}$. Any starting point will do. The example starts from zeros.
3. **For each training example** (x, t) :
 - Calculate the output: $\hat{y} = \text{sign}(\sum_i w_i x_i + b)$. With this deck's 0/1 outputs, "sign" means 1 if the total is at least 0, else 0.

- If $\hat{y} \neq t$, update the weights: $w_i^{new} = w_i^{old} + \eta(t - \hat{y})x_i$ and $b^{new} = b^{old} + \eta(t - \hat{y})$.

4. **Repeat** until convergence or a maximum number of iterations. "Convergence" means a full pass through all the examples with no update. One full pass is called an **epoch**.

And then the promise, called the **convergence theorem**: *if the data is linearly separable, the perceptron learning algorithm will converge in a finite number of steps*. In plain words: if some straight line separates the 1s from the 0s, this procedure will find such a line, and it will not take forever. (If no such line exists, it can run forever. Chapter 6 shows that case.)

Σ MATHS, STEP BY STEP

PLAIN MATHS

For one example with inputs $x_1, \dots, x_n$ and target t :

$$h = \sum_{i=1}^n w_i x_i + b, \quad \hat{y} = \begin{cases} 1 & h \geq 0 \\ 0 & h < 0 \end{cases}$$

If $\hat{y} = t$: do nothing. If $\hat{y} \neq t$, for every i :

$$w_i \leftarrow w_i + \eta(t - \hat{y})x_i \\ b \leftarrow b + \eta(t - \hat{y})$$

Why this direction? Suppose $t = 1$ and $\hat{y} = 0$, so h was too small. Then $t - \hat{y} = +1$ and every w_i grows by ηx_i . Next time, the same input gives a bigger total:

$$h_{new} = h_{old} + \eta \left(\sum_i x_i^2 + 1 \right) \geq h_{old} + \eta$$

So h moves up, towards firing. If $t = 0$ and $\hat{y} = 1$, the same numbers with a minus sign push h down.

MATRIX FORM

Treat the bias as weight w_0 on a fixed input $x_0 = 1$. Put everything in columns of length $n + 1$:

$$\tilde{\mathbf{x}} = [1, x_1, \dots, x_n]^\top \\ \mathbf{w} = [w_0, w_1, \dots, w_n]^\top$$

Then the guess and the update are each one line:

$$h = \mathbf{w}^\top \tilde{\mathbf{x}}, \quad \mathbf{w} \leftarrow \mathbf{w} + \eta(t - \hat{y}) \tilde{\mathbf{x}}$$

The second line updates all $n + 1$ numbers at once. The first entry of $\tilde{\mathbf{x}}$ is 1, so the bias moves by $\eta(t - \hat{y})$, exactly as in the plain version. A computer runs this as one vector addition per mistake.

Learning the NOT gate

Your slide 34 sets up the smallest possible problem. One input x_1 , one output. Two examples:

Example	x_1	t
Eg:1	0	1
Eg:2	1	0

The perceptron has two weights. The slide writes the perceptron equation as

$$h = w_0 x_0 + w_1 x_1, \quad \hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ 0 & \text{if } h < 0 \end{cases}$$

with $x_0 = 1$ always, so $h = w_0 + w_1 x_1$.

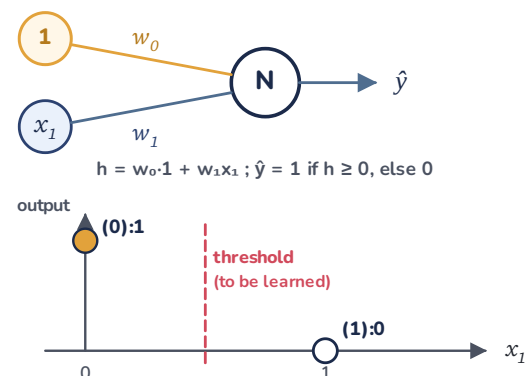


Figure 5.1 Slide 34's setup: the one-input perceptron, and the two examples on the x_1 line with the threshold that learning must place between them.

Settings (slide 35): assume $w_0 = w_1 = 0$, and let the learning rate $\eta = 1$. With $\eta = 1$ and 0/1 inputs, every update changes a weight by exactly +1, -1 or 0. The bias moves on *every* mistake, because $x_0 = 1$ is always on; w_1 moves only when $x_1 = 1$.

WORKED EXAMPLE: THE NOT GATE TRACE, EXACTLY AS ON SLIDES 35-39

Epoch 1. Weights $w_0 = 0, w_1 = 0$.

- 1 Example 1, $(x_1 = 0, t = 1)$: $h = w_1 x_1 + w_0 = 0 \cdot 0 + 0 = 0$. Since $h \geq 0$, $\hat{y} = 1$.
is $\hat{y}$ equal to t ? YES. No weight update needed.
- 2 Example 2, $(x_1 = 1, t = 0)$: $h = 0 \cdot 1 + 0 = 0$, so $\hat{y} = 1$.
is $\hat{y}$ equal to t ? ($\hat{y} = 1, t = 0$) NO. Update.
- 3 $w_1^{new} = 0 + 1(0 - 1) \cdot 1 = -1$.
 $w_i^{new} = w_i^{old} + \eta(t - \hat{y})x_i$ with $x_1 = 1$
- 4 $w_0^{new} = 0 + 1(0 - 1) = -1$.
the bias uses $x_0 = 1$

Epoch 2. Use the updated weights $w_1 = -1, w_0 = -1$.

- 1 Example 1, $(x_1 = 0, t = 1)$: $h = (-1) \cdot 0 + (-1) = -1$. Since $h < 0$, $\hat{y} = 0$.
is $\hat{y}$ equal to t ? ($\hat{y} = 0, t = 1$) NO. Update.
- 2 $w_1^{new} = (-1) + 1(1 - 0) \cdot 0 = -1$.
 $x_1 = 0$, so w_1 does not move
- 3 $w_0^{new} = (-1) + 1(1 - 0) = 0$.
the bias moves up by one
- 4 Example 2, $(x_1 = 1, t = 0)$: $h = (-1) \cdot 1 + 0 = -1$, so $\hat{y} = 0$.
is $\hat{y}$ equal to t ? YES. No update. Current weights $w_1 = -1, w_0 = 0$.

Epoch 3. Weights $w_1 = -1, w_0 = 0$.

- 1 Example 1, $(x_1 = 0, t = 1)$: $h = (-1) \cdot 0 + 0 = 0$, so $\hat{y} = 1$ (since $h \geq 0$).
YES, equal. No update.
- 2 Example 2, $(x_1 = 1, t = 0)$: $h = (-1) \cdot 1 + 0 = -1$, so $\hat{y} = 0$.
YES, equal. No update.
- 3 A whole epoch with no update: **the algorithm converges**. Final weights: $w_1 = -1, w_0 = 0$.

THE SAME TRACE AS ONE TABLE

Epoch	Ex	x_1	t	w_0	w_1	$h = w_0 + w_1 x_1$	$\hat{y}$	$\hat{y} = t?$	$t - \hat{y}$	new w_1	new w_0
1	1	0	1	0	0	0	1	yes	0	0	0
1	2	1	0	0	0	0	1	no	-1	-1	-1
2	1	0	1	-1	-1	-1	0	no	+1	-1	0
2	2	1	0	0	-1	-1	0	yes	0	-1	0
3	1	0	1	0	-1	0	1	yes	0	-1	0
3	2	1	0	0	-1	-1	0	yes	0	-1	0

Read a row left to right: the weights *before* the example, the total h , the guess, whether it was right, the correction $t - \hat{y}$, and the weights *after*. Highlighted cells are the only two moments anything changed. Epoch 3 has no "no", so learning stops.

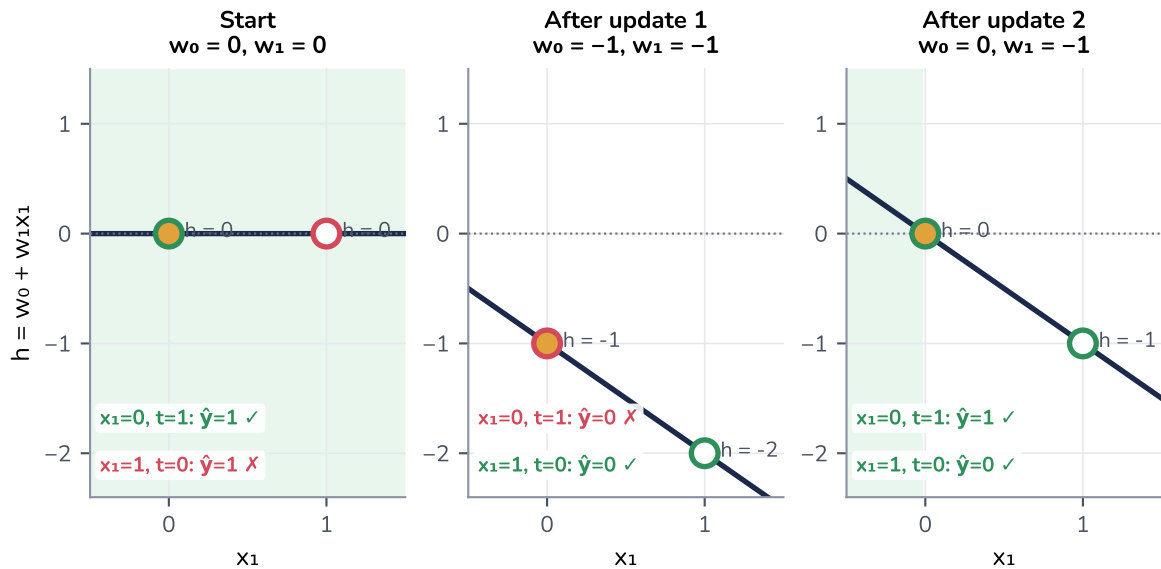


Figure 5.2 The line $h = w_0 + w_1 x_1$ after each change. Green is where $h \geq 0$, so the perceptron says 1. Orange dot: example 1 (target 1); white dot: example 2 (target 0); green ring = right, red ring = wrong. Start: $h = 0$ everywhere, both get 1, example 2 is wrong. After update 1 the line is below zero everywhere; now example 1 is wrong. After update 2 the line touches $h = 0$ exactly at $x_1 = 0$ (which counts as 1) and is -1 at $x_1 = 1$. Both right. The threshold has landed on $x_1 = 0$.

STOP AND THINK

Ravi points at epoch 2: "Example 1 was wrong, but w_1 stayed at -1 . Why did only the bias move?"

OUR ANSWER

Because example 1 has $x_1 = 0$. The update is $\eta(t - \hat{y})x_1 = 1 \cdot 1 \cdot 0 = 0$. An input that was switched off cannot be blamed for the mistake, so its weight is not touched. The bias input is always 1, so the bias always takes the correction. This is the "which knobs to turn" part of the volume analogy.

ON YOUR SLIDES

Slide 33 is the algorithm: "Initialize learning rate $\eta = 0.1$ ", "Initialize weights randomly", "For each training example (x, t) : calculate output $\hat{y} = \text{sign}(\sum_i w_i x_i + b)$; if $\hat{y} \neq t$, update $w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - \hat{y})x_i$, $b^{\text{new}} = b^{\text{old}} + \eta(t - \hat{y})$ ", "Repeat until convergence or max iterations", and the Convergence Theorem. Slide 34 is the NOT setup with $h = w_0 x_0 + w_1 x_1$ and a small picture: x_1 on a line, the point $(0):1$ on the left, $(1):0$ on the right, and a vertical "threshold" between them that learning must place. Slides 35–39 are the trace above, epoch by epoch, ending with "Algorithm Converges! Final weights: $w_1 = -1, w_0 = 0$ ". The slides write w_0 for the bias in the NOT example, but b in the algorithm; they are the same number.

CAREFUL

Three traps. (1) Use the guess $\hat{y}$ computed with the *old* weights; update only after you have compared it with t . (2) Go through the examples in the given order, one at a time; the answer can depend on the order. (3) "Converged" means a *whole* epoch with no update, not just one correct example. Epoch 2 above had one correct example and still needed epoch 3 to be sure.

🔗 TRY IT YOURSELF

Run the same NOT problem but present the examples in the other order: example 2 first, then example 1, still from $w_0 = w_1 = 0$ with $\eta = 1$. Which weights do you end with? (Epoch 1: ex 2 gives $h = 0, \hat{y} = 1 \neq 0$, so $w_1 = -1, w_0 = -1$; ex 1 gives $h = -1, \hat{y} = 0 \neq 1$, so $w_0 = 0$. Epoch 2: both right. Final $w_1 = -1, w_0 = 0$, the same line, reached in two epochs.)

✅ CHECK YOURSELF

1. Write the perceptron update rule. $w_i \leftarrow w_i + \eta(t - \hat{y})x_i; b \leftarrow b + \eta(t - \hat{y})$, **only when $\hat{y} \neq t$.**
2. What is $t - \hat{y}$ when the perceptron says 1 but the target is 0? **-1: the weights of active inputs go down.**
3. What does the convergence theorem promise? **If the data is linearly separable, the algorithm stops after a finite number of steps.**
4. In the NOT trace, what were the final weights and in which epoch did learning stop? **$w_1 = -1, w_0 = 0$; epoch 3 had no updates.**
5. Why did w_1 never go below -1? **Only example 2 has $x_1 = 1$, and it was wrong only once.**

🔗 IN ONE BREATH

- Guess, compare, correct: $w_i \leftarrow w_i + \eta(t - \hat{y})x_i, b \leftarrow b + \eta(t - \hat{y})$. Right answer, no change. Repeat until an epoch with no change.
- Vector form: $\mathbf{w} \leftarrow \mathbf{w} + \eta(t - \hat{y})\tilde{\mathbf{x}}$ with $\tilde{\mathbf{x}} = [1, x_1, \dots, x_n]^\top$.
- NOT gate, $\eta = 1$, start at 0: updates after epoch 1 example 2 ($\rightarrow -1, -1$) and epoch 2 example 1 ($\rightarrow w_0 = 0$); epoch 3 clean; final $w_1 = -1, w_0 = 0$.
- Linearly separable $\Rightarrow$ guaranteed to converge in finite steps.

The XOR wall

One perceptron can only draw one straight line, and some very small problems cannot be cut by a straight line.

On your slides: **Module 2, slides 41–42, 45–48, 50–51** · The XOR problem · Why a single perceptron fails · Linearly separable data · Perceptron for linearly separable data · Key concepts · Practice problems

📖 STORY

Ravi has an idea. "Make Nova show a 1 when exactly one of us is in the room. Then I know I can borrow your charger in peace." Two sensors: x_1 = Asha is in, x_2 = Ravi is in. Output 1 for (1, 0) and (0, 1), output 0 for (0, 0) and (1, 1). "That is an XOR gate," Asha says, and starts the learning algorithm from zero weights, like the NOT gate. Nova's screen shows the weights changing. And changing. Ten epochs. Fifty. They never settle. Asha draws the four points on the whiteboard, picks up a ruler, and tries to draw a straight line between the 1s and the 0s. She cannot.

! BIG IDEA

A perceptron draws one straight line and answers 1 on one side. Data that a straight line can split is called **linearly separable**. AND, OR, NOT are. **XOR is not**: its two 1s sit on one diagonal and its two 0s on the other, and no single line can keep them apart. So a single perceptron cannot learn XOR, and the learning algorithm never converges on it. The fix is more layers, and that is Book 5.

≈ THINK OF IT LIKE...

Four desks in a square. The two students on one diagonal are "in", the two on the other diagonal are "out". Try to lay the rope from Chapter 4, straight, so that only the "in" students are on one side. However you turn it, one desk is always on the wrong side. To do the job you need a rope that bends, or two ropes. A bent rope is what two layers of perceptrons give you.

The XOR problem

XOR means "exclusive or": 1 when the inputs are *different*, 0 when they are the *same*. Your slide 41 gives the table and draws the four points.

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0

The slide's verdict: **XOR is not linearly separable. A single perceptron cannot solve XOR.** Slide 42 explains why in one line: any line $w_1x_1 + w_2x_2 + b = 0$ cannot separate the XOR classes. *No single line works!* And the solution: **need multiple layers** (a multilayer perceptron).

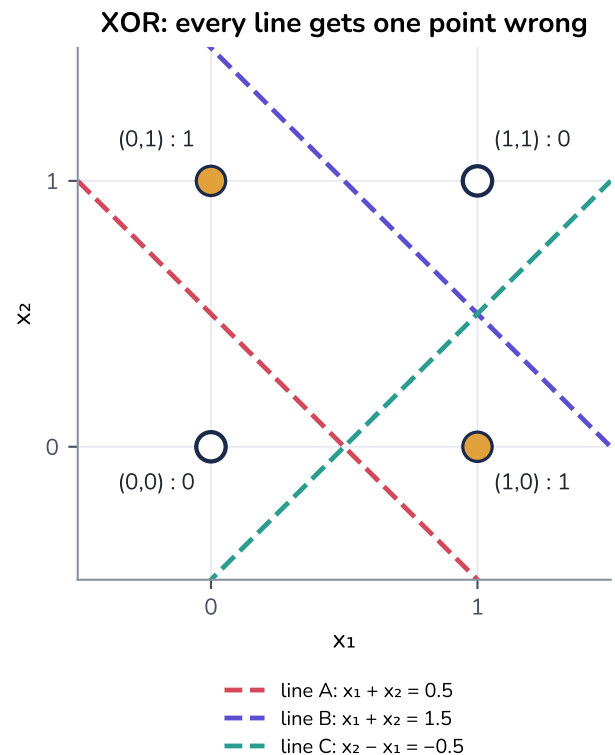


Figure 6.1 The four XOR points, like your slides 41–42. The 1s are on one diagonal, the 0s on the other. Three attempts at a line: A leaves (1,1) with the 1s, B leaves (0,0) with the 1s, C splits the 1s from each other. Every straight line gets at least one point wrong.

WORKED EXAMPLE: PROVING THAT NO LINE WORKS

Suppose some weights w_0, w_1, w_2 did the job. Write the four conditions, as in Chapter 4, with $h = w_0 + w_1x_1 + w_2x_2$:

- 1 (0,0) must give 0: $w_0 < 0$.
- 2 (0,1) must give 1: $w_0 + w_2 \geq 0$.
- 3 (1,0) must give 1: $w_0 + w_1 \geq 0$.
- 4 (1,1) must give 0: $w_0 + w_1 + w_2 < 0$.
- 5 Add lines 2 and 3: $2w_0 + w_1 + w_2 \geq 0$.
two "at least 0" things added are still at least 0
- 6 Add lines 1 and 4: $2w_0 + w_1 + w_2 < 0$.
two "less than 0" things added are still less than 0
- 7 Lines 5 and 6 say the same number is both ≥ 0 and < 0 . Impossible. So no such weights exist.
a contradiction: our supposition was wrong

This is why the learning algorithm never stops on XOR: it is searching for weights that do not exist. The convergence theorem only promised an end for linearly separable data.

Linearly separable data

Your slide 45 gives the general definition. Read it in three parts.

DEFINITION

Two sets of data points are **linearly separable** in an n -dimensional space if they can be separated by an $(n - 1)$ -dimensional **hyperplane**.

In 2-D space (two features), the hyperplane is a straight line. If a straight line can separate all the examples of class +1 from all the examples of class -1, the data is 2-D linearly separable.

An **infinite number** of straight lines can be drawn to separate the class +1 from the class -1.

- " n -dimensional" means n features. A "hyperplane" is the flat thing one dimension smaller: a line in 2-D, a flat sheet in 3-D, and so on. A perceptron with n inputs always draws exactly one such flat cut, because $h = 0$ is a linear equation in $x_1, \dots, x_n$.
- "Infinite number of lines": if one line works, you can shift or tilt it a little and it still works. That is why AND had the solution $(-1, 0.75, 0.75)$ and also $(-1.5, 1, 1)$. The learning algorithm finds *one* of them, not a special one.

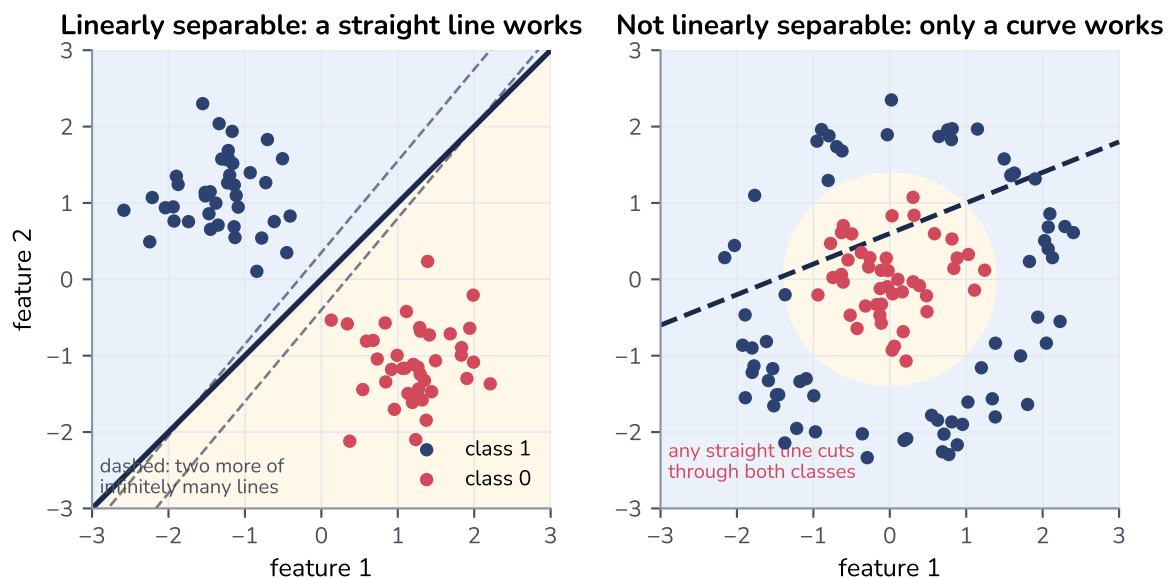


Figure 6.2 Like your slide 46. Left: two clouds that a straight line can split; the dashed lines are two of the infinitely many other lines that also work. Right: one class forms a ring around the other. Any straight line cuts through both classes, so this data is not linearly separable. Only a curved boundary works, and one perceptron cannot draw a curve.

The perceptron problem as four components

Your slide 47 rewrites everything in this book in the language of Book 1: the four parts of a learning problem. **Data**: a truth table, or any set of examples. **Model**: the perceptron; the inputs x are the features (the columns) and the target t is the required label. **Objective function**: the deviation between the desired output t and the computed output $\hat{y}$. **Learning algorithm**: the perceptron learning algorithm.



Challenge: how to find the $n + 1$ numbers $w_0, w_1, \dots, w_n$?

Answer: let the perceptron learning algorithm find them.

Figure 6.3 The four components for a perceptron, and the slide's "challenge" and "solution": the $n + 1$ parameters $w_0, \dots, w_n$ are learned by the perceptron learning algorithm.

Σ MATHS, STEP BY STEP (SLIDE 48)

PLAIN MATHS

With n inputs the perceptron computes

$$h = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

and then

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ 0 & \text{if } h < 0 \end{cases}$$

That is n multiplications, n additions, and one comparison. The $n + 1$ numbers $w_0, \dots, w_n$ are the whole model.

→

MATRIX FORM

Put a 1 in front of the inputs and the bias in front of the weights. The slide draws them as two tall columns:

$$\tilde{\mathbf{x}} = [1, x_1, x_2, \dots, x_n]^\top$$

$$\mathbf{w} = [w_0, w_1, w_2, \dots, w_n]^\top$$

Both are $(n + 1) \times 1$. Then the whole first line is one dot product:

$$h = \mathbf{w}^\top \tilde{\mathbf{x}}$$

The tilde on $\tilde{\mathbf{x}}$ reminds you that a 1 has been added at the top. This is the form the slide's "Key Concepts" use: $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$, where the bias is hidden inside $\mathbf{w}$.

Key concepts

Your slide 50 sums up the module. Here it is, with the chapter that explains each line.

Key concept (from the slide)	Where in this book
Neural network: computational model inspired by biological neurons that processes information through interconnected units	Chapters 1–3
Perceptron: single-layer linear classifier $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$	Chapter 4, and the ladder above
Learning rule: $w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - \hat{y})x_i$	Chapter 5
Converges for linearly separable data	Chapter 5 (theorem), Chapter 6 (definition)
Perceptron limitations: cannot solve XOR and other non-linearly separable problems	This chapter
Solution: multi-layer networks with non-linear activation functions	Book 5
Foundation: these concepts form the basis for deep learning architectures	Books 3–6

Practice problems

Your slide 51 asks you to represent the OR, AND, NOR and NAND gates with a perceptron and to *compute the parameters using the perceptron learning algorithm*. Here is the first one in full, with the NOT settings: $\eta = 1$, weights start at 0, examples in the order $(0, 0), (0, 1), (1, 0), (1, 1)$, outputs 0/1.

WORKED EXAMPLE: LEARNING THE OR GATE

Targets $t = 0, 1, 1, 1$; weights start at $(0, 0, 0)$. Each row: weights before, $h = w_0 + w_1x_1 + w_2x_2$, the guess, the weights after. A wrong guess adds $t - \hat{y}$ to w_0 , $(t - \hat{y})x_1$ to w_1 , $(t - \hat{y})x_2$ to w_2 .

Epoch	(x_1, x_2)	t	w_0	w_1	w_2	h	$\hat{y}$	ok?	new w_0	new w_1	new w_2
1	$(0, 0)$	0	0	0	0	0	1	no	-1	0	0
1	$(0, 1)$	1	-1	0	0	-1	0	no	0	0	1
1	$(1, 0)$	1	0	0	1	0	1	yes	0	0	1
1	$(1, 1)$	1	0	0	1	1	1	yes	0	0	1
2	$(0, 0)$	0	0	0	1	0	1	no	-1	0	1
2	$(0, 1)$	1	-1	0	1	0	1	yes	-1	0	1
2	$(1, 0)$	1	-1	0	1	-1	0	no	0	1	1
2	$(1, 1)$	1	0	1	1	2	1	yes	0	1	1
3	$(0, 0)$	0	0	1	1	0	1	no	-1	1	1
3	$(0, 1)$	1	-1	1	1	0	1	yes	-1	1	1
3	$(1, 0)$	1	-1	1	1	0	1	yes	-1	1	1
3	$(1, 1)$	1	-1	1	1	1	1	yes	-1	1	1
4	all four	0,1,1,1	-1	1	1	-1, 0, 0, 1	0,1,1,1	yes ×4	-1	1	1

Epoch 4 (last row, all four examples) has no update: converged after 5 updates. **Final weights** $w_0 = -1, w_1 = 1, w_2 = 1$, a different OR line from the slide's $(-1, 2, 2)$ and just as correct.

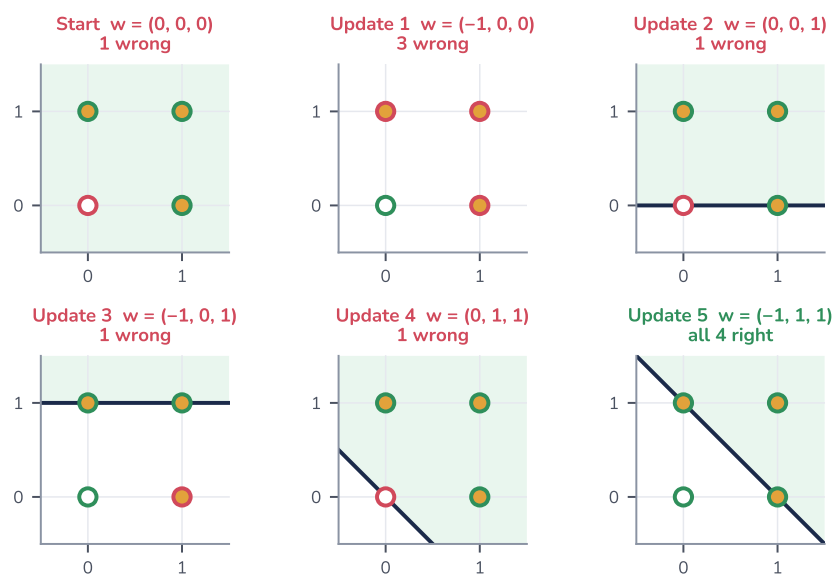


Figure 6.4 The OR boundary after each update. Green: the perceptron says 1; orange: target 1; white: target 0; red ring: wrong. The line settles at $-1 + x_1 + x_2 = 0$. A point on the line counts as 1 ($h = 0$ gives $\hat{y} = 1$).

THE OTHER THREE, FOR YOU TO DO

Same settings: $\eta = 1$, start at $(0, 0, 0)$, the same example order. Do them by hand, one row at a time, exactly like the table above. Our computed results, so you can check your final answer:

Gate	Targets for $(0,0), (0,1),(1,0),(1,1)$	Hint	Final (w_0, w_1, w_2)	Updates / epochs
AND	0, 0, 0, 1	Only $(1,1)$ pushes weights up; the other three push w_0 down. Expect a very negative bias.	$(-3, 2, 1)$	11 / 6
NOR	1, 0, 0, 0	The opposite of OR. Only $(0,0)$ is 1, so the bias must be at least 0 and both weights negative.	$(0, -1, -1)$	4 / 4
NAND	1, 1, 1, 0	The opposite of AND. Three points push up, only $(1,1)$ pushes down. Expect a positive bias and negative weights.	$(2, -2, -1)$	10 / 6

"Updates / epochs" counts how many times the weights changed and the epoch in which a clean pass happened. If you start from different weights, or use a different order, you will get different final weights. They are correct too, as long as all four rows come out right.

? STOP AND THINK

Ravi asks: "For XOR, can't Asha just use a bigger learning rate, or run more epochs?"

✓ OUR ANSWER

No. The learning rate only changes how far each step goes. More epochs only means more steps. But every step still lands on a straight line, and we proved that no straight line works. The problem is not the search; it is what is being searched. To beat XOR the model itself must change: more than one line, joined by a non-linear step in between. That is a multilayer network, Book 5.

📄 ON YOUR SLIDES

Slide 41 is the XOR truth table and the four-point picture, ending "XOR is not linearly separable! Single perceptron cannot solve XOR." Slide 42 is "Why Single Perceptron Fails on XOR" with the dashed lines, "No single line works!" and "Solution: Need multiple layers (Multilayer Perceptron)". Slide 43 links a Colab notebook (BITS email access only). Slide 45 is the linearly separable definition, slide 46 the two scatter plots, slide 47 the four components, slide 48 the vector form with $\tilde{\mathbf{x}}$ and $\mathbf{w}$ as columns and $h = \mathbf{w}^T \tilde{\mathbf{x}}$. Slide 50 is Key Concepts, slide 51 the four practice problems, slide 52 the references (d2l.ai chapter 1, Goodfellow et al. chapters 1–6, Rosenblatt 1958, Minsky and Papert 1969).

! CAREFUL

"Not linearly separable" does not mean "cannot be learned". It means one straight cut is not enough. In 1969 Minsky and Papert showed exactly this XOR limit, and interest in perceptrons faded for years. The way out was already there: stack perceptrons in layers. Also note: the learning algorithm never tells you that a problem is impossible; it simply keeps going. Always check whether a line can exist.

✓ CHECK YOURSELF

1. Which two XOR points have target 1? **(0,1) and (1,0), one diagonal.**
2. What does "linearly separable in n dimensions" mean, and how many separating lines exist?
Some $(n - 1)$ -dimensional hyperplane separates the classes (in 2-D, a line); infinitely many of them.
3. Write the perceptron in vector form. **$h = \mathbf{w}^\top \tilde{\mathbf{x}}$ with $\tilde{\mathbf{x}} = [1, x_1, \dots, x_n]^\top$; $\hat{y} = 1$ if $h \geq 0$, else 0.**
4. What happens if you run the perceptron learning algorithm on XOR? **It never converges; the weights keep changing.**
5. What is the slide's solution to the XOR limit? **Multiple layers with non-linear activations: a multilayer perceptron.**

● IN ONE BREATH

- XOR: 1 for (0,1) and (1,0), 0 for (0,0) and (1,1). No line works: the four conditions contradict each other.
- Linearly separable = one $(n - 1)$ -dimensional hyperplane (in 2-D, a line) splits the classes; infinitely many exist. Only then does the perceptron converge.
- Perceptron in one line: $h = \mathbf{w}^\top \tilde{\mathbf{x}}$, threshold at 0. Its $n + 1$ parameters are learned by the perceptron learning algorithm.
- OR learned from zero with $\eta = 1$: five updates, final $(-1, 1, 1)$. AND, NOR, NAND: do them yourself.
- One cell hits a wall at XOR. We need layers.

What comes next

Asha has one cell that can learn, and a wall it cannot cross. Layers fix the wall in Book 5. But first the cell needs two upgrades: answer "how much?" with a number instead of a hard yes/no, and learn from *how wrong* it was, not just *whether* it was wrong. That is Book 3, *Guessing How Much*, where Meena aunty asks Nova to guess the price of a flat.

Where you are in the series

Six small books, one story. Asha builds Nova, a robot that must learn to hear its name — and along the way she meets every idea in the Deep Neural Network course, in the order the course teaches them.

BOOK 1

Teaching a Machine to Listen

What deep learning is, and the four parts of every learning problem

BOOK 2

One Tiny Brain Cell

The artificial neuron, the perceptron, logic gates and the XOR wall

BOOK 3

Guessing How Much

A linear neuron for regression, squared loss and gradient descent

BOOK 4

Guessing Which One

Sigmoid and softmax neurons, cross-entropy, SGD and mini-batches

BOOK 5

A Real Brain

Hidden layers, activations, backpropagation by hand, and design

BOOK 6

Eyes for the Machine

Convolution, feature maps, pooling, famous CNNs and transfer learning

Next: **Book 3, Guessing How Much** — one neuron that answers "how much?", the squared loss, and gradient descent with real numbers.